Benchmarking and Fine-tuning Computer Vision Models for Object Detection Tasks

Neha Naveen, Anya Ross, Mohamed Zakaria Kheder, Spurthi Setty, Zexun Yao, Alan Zhao, Rishav Acharya, Ani Jain, Natan Vidra

Abstract

This paper benchmarks several computer vision models and fine-tuning approaches for object detection tasks across diverse datasets in order to understand how various architectures and fine-tuning strategies impact performance on specialized tasks relating to object detection and classification in field settings. We evaluate both zero-shot and fine-tuned performance on datasets from different domains, including underwater imagery and tactical surveillance-style data. Of the models fine-tuned to the data, the YOLOv8 architecture showed the most promise with respect to accuracy in classification.

1 Introduction

Object detection is a cornerstone task in computer vision with applications in autonomous vehicles, robotics, and marine biology. Although general-purpose models have shown high accuracy in COCO datasets, their performance often degrades in more domain-specific environments. This motivates our investigation to benchmark various detection models and understand the effectiveness of fine-tuning strategies to improve performance on specialized data. We explore how zero-shot transfer compares to targeted adaptation via fine-tuning, and we assess how different training configurations impact model accuracy and robustness in new environments.

2 Background

2.1 History of Object Detection

Early methods relied on hand-crafted features such as HOG and SIFT combined with sliding window approaches. These techniques were computationally expensive and did not scale well to complex scenes or diverse object types. The advent of deep learning introduced models like Faster R-CNN, YOLO, and DINO, enabling real-time and accurate detection. In this paper, we explore the following models.

1. *DINO (DETR-based)*: Transformer-based, end-to-end detectors suitable for complex object categories.

2. *Faster R-CNN*: A two-stage detector with high accuracy, especially in high-resolution or cluttered scenes.

3. *YOLOv8*: Known for speed and simplicity, ideal for real-time applications.

2.2 Zero-shot vs Fine-tuned model performance

Zero-shot models are evaluated without additional training on the target dataset. Fine-tuning models on the dataset updates the model weights to match specific data distributions, typically yielding better performance at the cost of additional training.

2.3 Evaluation metrics

The results from each experiment was evaluted using the following metrics.

1. *mAP* (mean Average Precision): Aggregates detection quality across IoU thresholds and object classes.

2. *IoU* (Intersection over Union): Measures overlap between predicted and ground truth bounding boxes.

3. *Precision/Recall*: Key indicators for balancing false positives and false negatives, especially in critical applications.

3 Datasets

We selected these three datasets—Military Aircraft, TrashCan, and FAIR1M—because they span a wide variety of real-world image domains, including aerial, underwater, and satellite imagery. This diversity allows us to evaluate model performance across different environments, object types, and levels of class imbalance, providing a comprehensive benchmark for object detection robustness.

3.1 Military Aircraft

The Military Aircraft Dataset is a comprehensive collection of 81 different kinds of aircraft types for object detection tasks. This dataset is available on kaggle, and was used to finetune and train the YOLOv8 model. It has over 20,000 images and was divided into a 80/5/15 train/validation/test split.

A variant of this dataset available on hugging face in COCO format was used to evaluate DINO and Faster-R-CNN. This dataset contains a substantial collection of aerial images featuring 20 distinct classes of military aircraft, labeled from A1 to A20. It contains 2,907 images in the training split, 330 images in

validation, and 605 images in the test split. The class distribution is moderately imbalanced: for example, class A16 is the most frequent with over 2,300 images across splits, while smaller classes like A18 and A11 have fewer than 300 images total. Most classes have several hundred images, with common classes such as A14, A2, and A13 ranging between approximately 1,200 to 1,300 images in training alone. This distribution supports both fine-grained classification and detection tasks, but the imbalance suggests that models may benefit from class-weighting or augmentation strategies to improve performance on less frequent classes.

This dataset, similar to the MTARSI dataset in scope, is suited for defense and surveillance applications, enabling detailed recognition of varied military aircraft types. The wide range of class sizes reflects real-world operational distributions, offering an authentic challenge for automated aerial target recognition.

3.2 FAIR1M

FAIR1M is a high-resolution remote sensing dataset for fine-grained object recognition known for its diversity and oriented bounding boxes. The original dataset contains over 1 million annotations across 15,000 images, however subsets of this were used to train the different models. The YOLO model was trained on a Kaggle dataset with over 15,000 labeled examples. Faster R CNN and DINO used a Hugging Face dataset in COCO format based off of FAIR1M, containing 1,732 images split into 1,402 for training, 156 for validation, and 174 for testing. It features a large number of object instances with significant class imbalance. The training split includes dominant classes such as Small Car (26,000 instances) and Van (24,500), while medium-frequency classes like Dump Truck (3,700) and Cargo Truck (2,200) provide moderate representation. The dataset also contains a variety of aircraft types like Boeing737 (295), A220 (372), and A321 (202), as well as ship categories including Dry Cargo Ship (1,508) and Fishing Boat (428).

Smaller classes, such as Warship (78), Tractor (64), and specialized aircraft like A350 (5) and C919 (3), have far fewer instances, reflecting real-world rarity. This wide distribution supports development of models capable of recognizing both common and rare objects in complex urban and maritime remote sensing scenes, useful for applications in urban planning, environmental monitoring, and defense.

3.3 Trashcan

The TrashCan dataset focuses on underwater visual detection of marine debris and associated marine life, providing 7,212 total images split into 4,989 for training, 649 for validation, and 1,574 for testing. The dataset covers a broad spectrum of classes, including multiple trash categories like plastic (1,304 images in training), metal (748), wood (243), fabric (194), paper (134), and fishing

gear (118), as well as diverse marine animals such as fish (586), starfish (224), crabs (138), eels (264), and others. The training split shows a dominant class 'rov' (Remotely Operated Vehicle) with 2,679 images, which appears consistently across splits.

Class imbalance is evident, with some classes like trash-rubber containing fewer than 10 images in training, while others have thousands. The dataset's separation by videos ensures no temporal overlap between splits, promoting reliable generalization evaluation. This dataset is valuable for environmental monitoring and robotics applications where precise detection of underwater debris amidst marine life is critical.

4 Methodology

4.1 Baseline (Zero-Shot) Inference

Each model was tested on raw datasets using pre-trained weights (e.g., trained on COCO or ImageNet) without any additional fine-tuning or dataset-specific modifications. Although the original data was given as a list of total annotations where an image could have several different annotations, zero shot predictions were conducted on a per image rather than per annotation basis.

4.1.1 DINO

The zero-shot evaluation was performed using Grounding DINO, a model built on top of the DINO Vision Transformer, enhanced to support open-vocabulary object detection by grounding textual prompts in visual inputs. Although trained in large-scale datasets with broad visual-text associations, Grounding DINO was not fine-tuned in our domain-specific aircraft categories, which limits its ability to deliver accurate predictions for our task.

In our experiments, we observed that increasing the number of text prompts—each representing a specific class—led to a sharp decline in detection confidence. As more prompts were added, the model's attention became diluted across categories, resulting in lower confidence scores overall. This trade-off posed a challenge: setting a high confidence threshold filtered out nearly all detections, while lowering it introduced excessive noise and false positives by labeling nearly every region.

To address this, we adopted a two-step detection pipeline. First, we used a broad category prompt such as "aircraft" to acquire coarse bounding boxes of relevant regions. Then, we cropped these regions and re-ran Grounding DINO with a smaller, more specific set of prompts (e.g., "A321", "Boeing737" etc.). This approach helped isolate contextually relevant areas and improved the focus of subsequent class predictions.

However, even with this workaround, the model's zero-shot predictions lacked consistency and precision, particularly when dealing with fine-grained distinctions between aircraft types. These results underscore the limitations of relying solely on pre-trained Grounding DINO for specialized domains and highlight the need for fine-tuning or incorporating supervised data with aligned class annotations to achieve robust and interpretable results.

4.1.2 Faster R-CNN

The zero shot evaluations for Faster R CNN was conducted using a pre-trained Faster R-CNN model from the torchvision library. This model has a ResNet-50 backbone and is pretrained on the COCO dataset. After the data is preprocessed and loaded in, for each unique image, it is passed through the model to collect the predicted bounding boxes, class labels and associated confidence scores.

Since the datasets used have labels outside the scope of the pretrained model, the confidence scores of these zero shot predictions were very low. In order to ensure that a fair amount of predictions were being made, predictions were filtered based on a confidence threshold of 0.10, as most of the predictions had less than 0.20 probability. This posed a similar tradeoff to the DINO model where a high confidence threshold risked filtering out most of the predictions while a low one kept a lot of false positives low quality predictions.

4.1.3 YOLOv8

The YOLO zero-shot evaluation was conducted using the YOLOv8 model pretrained exclusively on the COCO dataset, which contains annotations for only 80 object categories. Since our datasets include classes that do not have direct counterparts within the COCO label set, the pretrained YOLOv8 model is inherently limited in its ability to correctly classify these specialized categories. As a result, when employing zero-shot inference, the model fails to produce accurate class predictions specific to our datasets.

Although the YOLOv8 model may detect and localize objects that visually resemble those in our datasets, it cannot assign the correct fine-grained labels because its training scope does not cover our unique classes. This discrepancy leads to a situation where object detections may be semantically related but are ultimately mismatched in terms of class identification. Therefore, zero-shot predictions from this pretrained model do not yield meaningful or reliable results for our domain-specific tasks, highlighting the necessity for fine-tuning or training on datasets with corresponding class annotations to achieve precise recognition.

5 Fine-Tuning Strategies

Fine-tuning is a critical step in adapting pre-trained object detection models to specific target domains or datasets. It involves adjusting the model parameters, partially or fully, based on new training data. The goal is to retain useful learned features from large-scale pre-training (e.g., on COCO) while enabling the model to specialize in the new task. This is essential to improve accuracy and reliability for specific types of images and labels that are not commonly found in pre-training datasets. Common fine-tuning approaches include:

1. *Full Fine-Tuning:* All model parameters are updated during training. This provides the most flexibility and performance gains but is computationally intensive.
2. *Transfer Learning with Frozen Backbone:* Only task-specific layers (e.g., detection heads) are trained, while the backbone is frozen. This reduces overfitting and training cost.
3. *Lightweight methods (e.g., LoRA):* Parameter-efficient tuning strategies that adapt a subset of weights, offering a trade-off between performance and efficiency.

In our experiments, we fine-tuned each model separately according to best practices and available options for that architecture. Due to compute restraints we finetuned YOLOv8 on a subset of two of the datasets, and Faster R-CNN on one dataset.

5.1 Faster R-CNN Fine-Tuning

Faster R-CNN was fine-tuned using the pre-trained `fasterrcnn_resnet50_fpn` model from the TorchVision library. The classification head was modified to match the number of target classes in the dataset (plus one for the background). Training was performed end-to-end with all layers unfrozen, enabling full fine-tuning.

Model Configuration

- Base model: `fasterrcnn_resnet50_fpn` (pre-trained on COCO)
- Predictor head: replaced with `FastRCNNPredictor` for $N + 1$ classes

Optimization Setup

- Optimizer: SGD with momentum 0.9
- Learning rate: 0.0025
- Weight decay: 1×10^{-4}
- Learning rate scheduler: step decay with step size 5 and gamma 0.1

Training Details

- Number of epochs: 3 (initial testing; extendable)
- Loss: total of values returned in the model's loss dictionary
- Backpropagation: performed on total loss
- Device: trained on GPU if available
- Learning rate step: updated after each epoch

5.2 YOLOv8 Fine-Tuning

YOLOv8 was fine-tuned using a **full fine-tuning** approach. All model parameters—including the backbone, RPN modules, and detection heads—were updated using Stochastic Gradient Descent (SGD) with momentum 0.9 and weight decay 1×10^{-4}.

Training Details

- Epochs: 100
- Batch size: 16
- Learning rate: 0.01
- Optimizer: SGD
- Learning rate schedule: reduced by a factor of 10 after epoch 10

Data Augmentation

- Mosaic augmentation
- Random affine transformations
- HSV (hue, saturation, value) color space shifts
- Horizontal flipping
- Scaling and cropping

Training Details
During inference, predictions with confidence scores above 0.2 were retained. Non-Maximum Suppression (NMS) with an IoU threshold of 0.3 was applied. Final detections were obtained by cropping proposals via ROI-Align into 7×7 feature grids, followed by classification and bounding box refinement.

Performance Note
Confidence scores ranged from approximately 0.21 to 0.74. Further improvements were explored by adjusting anchor configurations, applying targeted augmentations, and raising score thresholds.

6 Results and Evaluation

6.1 Performance Summary

This section displays the overall results of the experiments conducted and a few select confusion matrices and example predictions.

Model	DINO			Faster R-CNN				YOLOv8				
Training		Zero-Shot		Zero-Shot			Fine-Tuned	Zero-Shot			Fine-Tuned	
Dataset	FAir1M	Aircraft	Trashcan	FAir1M	Aircraft	Trashcan	Aircrafft	FAir1M	Aircraft	Trashcan	FAir1M	Aircraft
Precision:	0.0049	0.0018	0.0168	0.0002	0.0148	0.0064	0.2097	0	0	0	0	0
Recall:	0.0005	0.0017	0.031	0.0001	0.0531	0.0229	0.1568	0	0	0	0	0
F1:	0.0009	0.0018	0.0218	0.0002	0.0231	0.01	0.1794	0	0	0	0	0
Mean_iou:	0.6577	0.8092	0.8381	0.8222	0.7448	0.7203	0.6631	0	0	0	0	0
Map:	0.0072	0.0001	0.0031	0.0025	0.0034	0.0032	0.2037	0	0	0	0	0
Tp:	4	6	57	1	187	42	552	0	0	0	0	0
Fp:	820	3278	3328	4510	12450	6544	2080	197	87	419	84	503
Fn:	7880	3515	1779	7883	3334	1794	2969	7884	3521	1836	7884	3521

Table 1: Model performance metrics across different datasets and training types

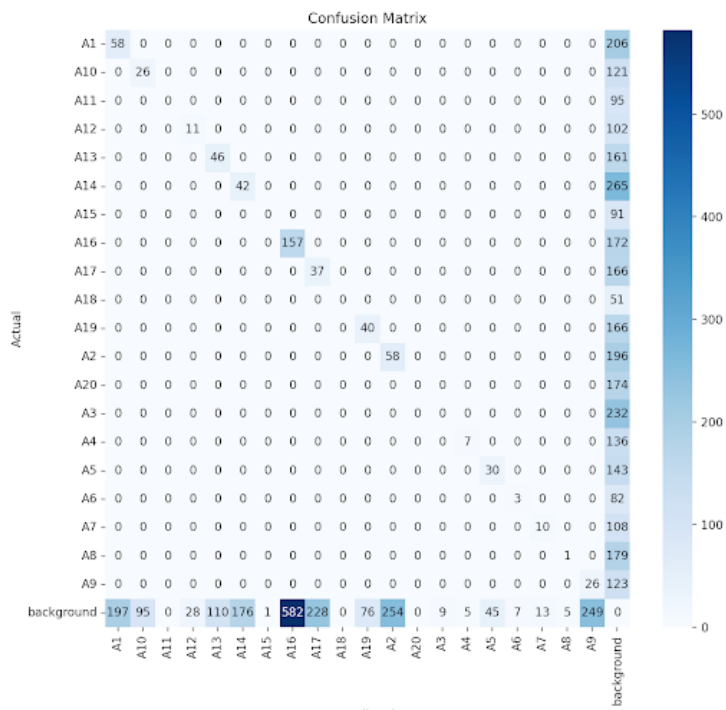

Figure 1: Finetuned Faster R-CNN Confusion Matrix for Aircraft Dataset

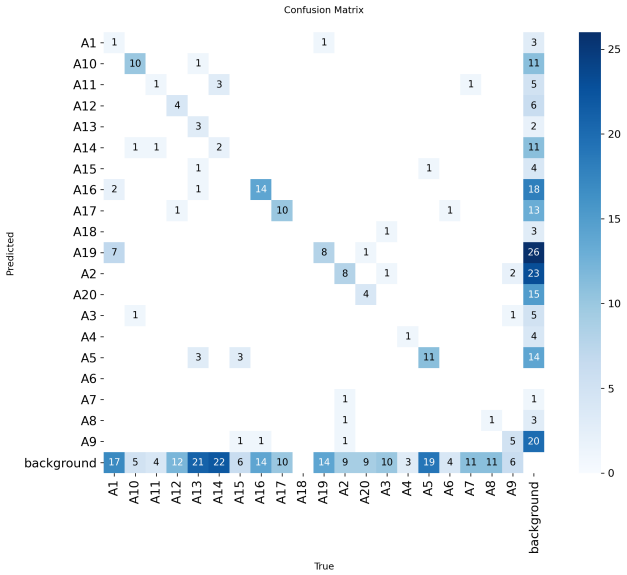

Figure 2: Finetuned YOLOv8 Confusion Matrix for Aircraft Dataset

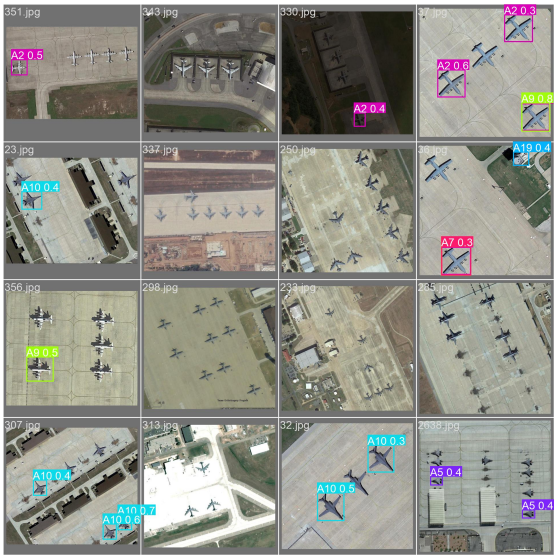

Figure 3: Example Aircraft Predictions for Finetuned YOLOv8

7 Discussion

This section compares zero-shot versus fine-tuned performance across all datasets. In the underwater domain, fine-tuned models outperformed zero-shot detectors, particularly when dealing with occlusion or camouflage. Dense scenes with many small object instances benefited significantly from targeted fine-tuning. The pros and cons of different fine-tuning approaches are also discussed.

Grounding DINO proved to be the most challenging to adapt to the data. Due to the text-prompted nature of the model's training, it could not be fine tuned to the subtlety of the distinctions within the relevant image datasets. The model is able to process images and label with native classes and language, but is difficult with fine grained and specific tasks. This generality made the Grounding DINO model disqualified as a good benchmark.

The Faster R-CNN architecture was preferable to traditional R-CNN due to its efficient use of computing resources and faster training and evaluation times. The zero shot model performed poorly across all three datasets. The fine tuned model showed improvement, but was still unable to confidently identify all objects within an image. The model was not confident in assigning the labels of the Aircraft dataset. In particular images where more than two types of aircraft were present, the model had difficulty labeling more than one.The Faster-RCNN model evaluates images in regions, and this limits its accuracy in these more diversified classes. This model showed promise.

YOLOv8 is a highly efficient and accurate object detection model, making it particularly well-suited for real-time applications and scenarios with limited labeled data. Its architecture is optimized for speed, allowing fast inference while maintaining strong performance across a range of detection tasks. With support for transfer learning, YOLOv8 can be fine-tuned effectively using small datasets, thanks to robust pretraining and built-in data augmentation strategies. This combination of speed, adaptability, and ease of deployment makes YOLOv8 an ideal choice for practical, real-world applications where computational resources and labeled data are constrained.

It is important to know that all models were run on pure images. No preprocessing was done on the images, as the goal for the best model is to be able to process noisy data and classify efficiently. Noise reduction and preprocessing techniques would likely improve accuracy across all datasets and models. Thresholding, for example, would prove to distinguish the reflected light from aircrafts captured with a standard camera, eliminating the background noise. Additionally, the complex nature of the models limited which could be run due to constraints of technology, particularly GPU access.

8 Conclusion

We benchmarked several object detection models under zero-shot and fine-tuned settings across three datasets. Our results confirm that fine-tuning yields substantially better results in domain-specific tasks. YOLOv8 showed strong real-time performance, while DINO benefited more from fine-tuning in complex scenes. Model selection and adaptation should therefore be guided by application-specific needs, particularly in areas that have visual variability.

9 Future Work

Some future directions include:

- Exploring larger vision-language models (such as OWL-ViT or Grounding DINO) for better zero-shot grounding.
- Generating synthetic data using simulations or increase augmentation to support more few-shot domains.
- Incorporating active learning to selectively annotate and fine-tune with max gains.
- Optimizing for edge detection by pruning/introducing dropout and real-time performance benchmarks.

References

[1] Nicolas Carion et al. End-to-end object detection with transformers. In *European Conference on Computer Vision (ECCV)*, 2020.

[2] DigitalOcean. Faster r-cnn explained – object detection. `https://www.digitalocean.com/community/tutorials/faster-r-cnn-explained-object-detection`, 2023.

[3] Hugging Face. Object detection. `https://huggingface.co/docs/transformers/en/tasks/object_detection`, 2023.

[4] Edward Hu et al. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.

[5] Kaggle. Fair1m: Satellite imagery for object detection. `https://www.kaggle.com/datasets/ollypowell/fair1m-satellite-imagery-for-object-detection`, 2023.

[6] Kaggle. Military aircraft detection dataset. `https://www.kaggle.com/datasets/a2015003713/militaryaircraftdetectiondataset/data`, 2023.

[7] Tsung-Yi Lin et al. Microsoft coco: Common objects in context. In *ECCV*, 2014.

[8] Sajid Mim et al. Object detection models and evaluation metrics: A survey. *arXiv preprint arXiv:2103.05569*, 2021.

[9] Nandini Reddy. Implementing yolov8 in detail for beginners. `https://medium.com/@nandinilreddy/implementing-yolov8-in-detail-for-beginners-9a5d3b0fe30a`, 2023.

[10] Joseph Redmon et al. You only look once: Unified, real-time object detection. *arXiv preprint arXiv:1506.02640*, 2016.

[11] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems*, 2015.

[12] Rishabh Robu. Understanding and implementing faster r-cnn. `https://medium.com/@RobuRishabh/understanding-and-implementing-faster-r-cnn-248f7b25ff96`, 2023.

[13] Amit Shrivastava. Fine-tuning yolov8: A practical guide. `https://medium.com/@amit25173/fine-tuning-yolov8-a-practical-guide-61343dada5c1`, 2023.

[14] Augmented Startups. Yolov8 vs yolov5: Choosing the best object detection model. `https://www.augmentedstartups.com/blog/yolov8-vs-yolov5-choosing-the-best-object-detection-model`, 2023.

[15] Ultralytics. Yolov8 overview. `https://docs.ultralytics.com/models/yolov8/#overview`, 2023.

[16] YOLOv8.org. How to use & fine-tune yolov8. `https://yolov8.org/how-to-use-fine-tune-yolov8/`, 2023.

[17] Xizhou Zhu et al. Dino: Detr with improved denoising anchor boxes for end-to-end object detection. *arXiv preprint arXiv:2203.03605*, 2022.